

How Does a Java Program Execute?

Beginner-Friendly Deep Guide • From .java File to Final Output

In one line: A Java program is written in a .java file, compiled by javac into bytecode (.class), and then run by the JVM, which converts that bytecode into machine-level actions to produce output.

What you will learn

- What happens after you write Java code
- Why we need javac, bytecode, JVM, class loader and JIT
- The exact step-by-step execution flow
- Hands-on commands to compile and run Java
- Common errors and how to fix them
- Exam, viva and interview answers

Step 1	Step 2	Step 3	Step 4	Step 5
Write Code	Compile	Get Bytecode	JVM Runs	Output
.java	javac	.class	Class Loader + JIT	Console / App

1. Big Picture

Think of Java execution like this: you write instructions in a human-readable form, a compiler changes them into a special middle form called bytecode, and then the Java Virtual Machine (JVM) understands and runs that bytecode on your computer.

Simple real-life analogy

- You = the programmer
- .java file = your written instructions
- javac = translator
- .class file / bytecode = translated common language
- JVM = interpreter + execution manager
- Output = final result shown on screen

Core terms you must remember

Term	Meaning
.java file	Source code written by the programmer
javac	Java compiler that converts source code into bytecode
.class file	Compiled bytecode file
JVM	Java Virtual Machine that runs bytecode
JRE	Runtime environment needed to run Java programs
JDK	Developer kit that contains compiler and tools
JIT	Just-In-Time compiler that speeds up execution

2. Step-by-Step Execution Flow

Step 1: Write source code

You write Java code in a file such as Hello.java.

Step 2: Save the file

The filename should normally match the public class name.

Step 3: Compile with javac

The command `javac Hello.java` checks syntax and converts source code into bytecode.

Step 4: Generate .class file

If compilation is successful, Java creates Hello.class.

Step 5: Start JVM with java command

The command `java Hello` starts the JVM and asks it to run the class.

Step 6: Class Loader loads the class

The JVM brings the required class files into memory.

Step 7: Bytecode verification

The JVM checks whether the bytecode is valid and safe.

Step 8: Execution engine runs code

The interpreter and JIT compiler execute the bytecode.

Step 9: Memory management happens

Objects are created in memory and later cleaned by the garbage collector when not needed.

Step 10: Output appears

The program result is shown on the console, window, file, network, database, or any target used by the program.

3. Hands-On Tutorial

Example program

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```

How to compile and run

- 1) Open terminal / command prompt
- 2) Go to the folder where Hello.java is saved
- 3) Compile:
javac Hello.java
- 4) Run:
java Hello

What happens here?

- javac Hello.java creates Hello.class
- java Hello starts the JVM and runs the compiled class
- The JVM looks for the main method
- Then the statement System.out.println prints the output

Expected output

Hello, Java!

Important rule

When running the program, do not write java Hello.java. Normally you write only the class name: java Hello.

4. What Happens Inside the JVM?

The JVM has several important parts.

Part	Job
Class Loader	Loads .class files into memory
Bytecode Verifier	Checks safety and correctness of bytecode
Runtime Data Areas	Stores method data, stack, heap, etc.
Execution Engine	Runs bytecode using interpreter and JIT compiler
Garbage Collector	Removes unused objects from memory

Interpreter vs JIT

- Interpreter reads and runs bytecode instruction by instruction.
- JIT compiler converts frequently used bytecode into native machine code for faster execution.
- Because of JIT, Java becomes much faster after startup.

Memory areas in simple words

- Stack: stores method calls and local variables
- Heap: stores objects created using new
- Method Area / Metaspace: stores class-level information
- PC Register: keeps track of the current instruction

5. Why Java Is Platform-Independent

Java source code is first converted into bytecode. This bytecode is not tied to one operating system. Different operating systems have their own JVM implementation, so the same bytecode can run on Windows, Linux and macOS.

Flow

```
Java Source Code (.java)
    ↓
Compiler (javac)
    ↓
Bytecode (.class)
    ↓
JVM on Windows / Linux / macOS
    ↓
Program Output
```

Remember this sentence

Write Once, Run Anywhere means you compile once to bytecode, and a suitable JVM can run it on multiple platforms.

6. Common Errors and Fixes

- javac is not recognized → JDK may not be installed correctly or PATH is not set.
- Could not find or load main class → You may be in the wrong folder, class name may be wrong, or classpath issue exists.
- Class name does not match filename → Public class name and file name should match.
- Compilation error → There may be a syntax mistake like missing semicolon or bracket.
- No main method found → The JVM needs the correct main method to start a normal Java application.

7. Exam, Viva and Interview Answers

Short answer: How does a Java program execute?

A Java program is written in a .java file. The javac compiler converts it into bytecode stored in a .class file. Then the java command starts the JVM. The JVM loads the class, verifies the bytecode, and executes it using the interpreter and JIT compiler. Finally, the program produces output.

Very short viva answer

Source code → compiled by javac → bytecode → run by JVM → output.

Top interview questions

- What is bytecode?
- What is the role of javac?
- What is the difference between compilation and execution?
- What does the JVM do?
- What is the role of the class loader?
- What is JIT compilation?
- Why is Java called platform-independent?

8. Practice Tasks

- Write a program that prints your name and college.
- Compile it using javac and run it using java.
- Deliberately make a syntax error and observe the compiler message.
- Change the public class name and file name to different values and see the error.
- Explain the full execution flow in your own words without looking at this guide.

One-line revision sheet

Write code → save as .java → compile with javac → get .class bytecode → run with java → JVM loads + verifies + executes → JIT speeds it up → output appears

Final memory trick

Source → Compiler → Bytecode → JVM → Output

Prepared as a beginner-friendly audience handout for the topic: How does a Java program execute?